MD-Bench on Cuda
GPU port optimization

by: Martin Bauernfeind

advised by: Rafael Ravedutti

Project Description

Project description

Application Info

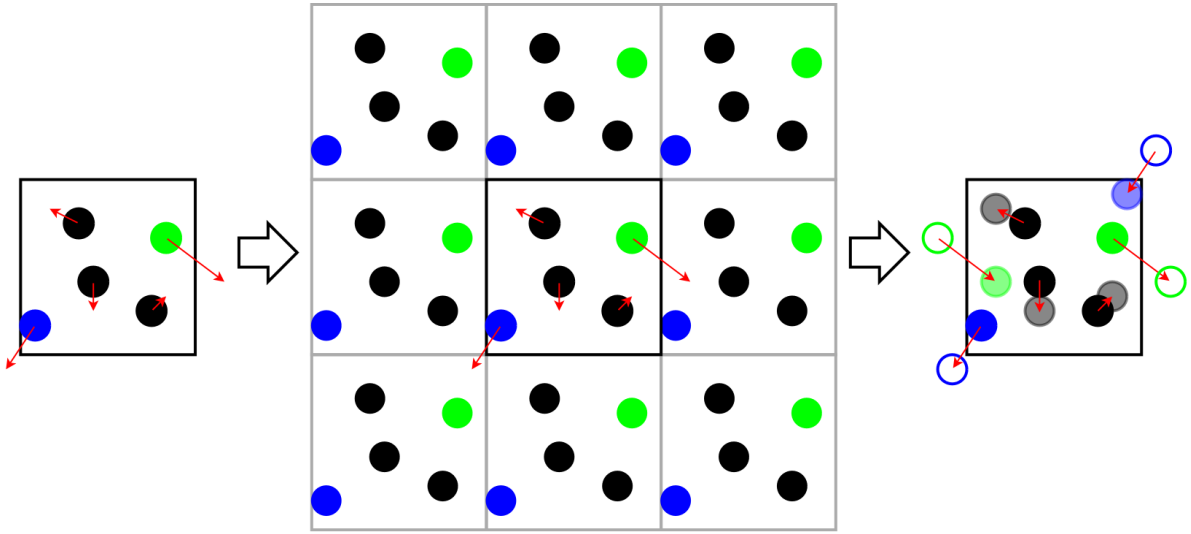
- original MD-Bench:
  - mini-app to simulate molecular dynamics
  - written in sequential C with less than 1000 lines of code
- MD-Bench on Cuda:
  - port MD-Bench to Cuda to use GPGPU parallelism
  - significant parts not ported yet
- Code: MD-Bench on Cuda
- URL: https://github.com/RRZE-HPC/MD-Bench/tree/mucosim_cuda

Project Description

Simulation Model

Molecular dynamics emulated in MD-Bench by:

- interaction among particles
- affects of these interactions on their motion
- constituents of simulation:
  - Number of atoms with initial state (position & velocity)
  - Boundary conditions (periodic)



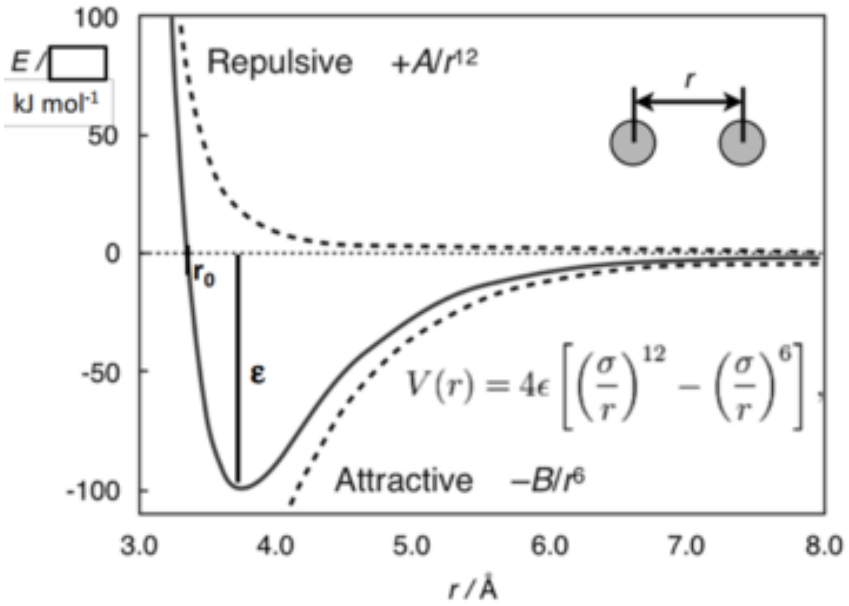
Note: The words particle and atom will be used interchangeably

Project Description

Particle interactions

force computation of particle interactions:

- based on the Lennard-Jones potential:
  - pairs of particles, here: electronically neutral atoms
  - models repulsive as well as attractive interactions



- ***r*** is the distance between the two interacting atoms,
- ϵ is the dispersion energy and
- σ the distance at which the particle-potential ***V*** is zero

Project Description

Simulation Process

The simulation runs similar to the sketch code below:

- every timestep iterate over all particles:
  - compute interactions with neighbors
  - integrate force generated by interaction

```
for t in timesteps:
  for atom in atoms:
    velocity[atom] += force[atom]
    position[atom] += velocity[atom]
  if t % 20 == 0:
    neighbors = calculateNeighbors()
  for atom in atoms:
    neighbors = neighbors[atom]
    force = 0.0
    for neighbor in neighbors:
        radius = calc_radius(...)
        if radius < close_enough:
            force += calc_force(...)
    forces[atom] += force
```

Some parts already have been ported to GPU - more on that later

Testsystem

Testsystem

- Host/Clustername: alex
- multiple nodes with multiple GPUs per node
- resources per GPU:
  - [A40 | A100]
  - CPU: [16 cores / GPU | 16 cores / GPU]
  - Memory: [60GB / GPU | 120GB / GPU]
- computational power
  - SP: [A40: 37.4 TFlops | A100: 19.5 Tflops]
  - DP: [A40: 0.5 TFlops | A100: 9.7 Tflops]

Note: each an A40 has about double the single precision processing power of an A100 despite being cheaper

Execution and default parameters

Executing the simulation

Software Environment

- Compiler: NVCC (cuda/11.6.1)
- Operating System: Ubuntu 20.04.3 LTS
- Addition libraries:
  - LIKWID 5.2.0

How to build software

```
$ git clone https://github.com/RRZE-HPC/MD-Bench/
$ cd MD-Bench
$ git checkout mucosim_cuda
$ module load likwid cuda
$ make
```

How to run software (e.g. in a batch job)

```
$ module load likwid cuda
$ cd MD-Bench
$ ./MDBench-NVCC
```

Testcase description

If not stated otherwise these are the conditions for the simulation benchmarking.

- the number of simulated atoms is 131072
- floating numbers use double precision
- force computation between atoms uses the lennard-jones model
- one gpu with its associated cores are used for computation

Profiling

Profiling

Initial: CPU Scaling runs

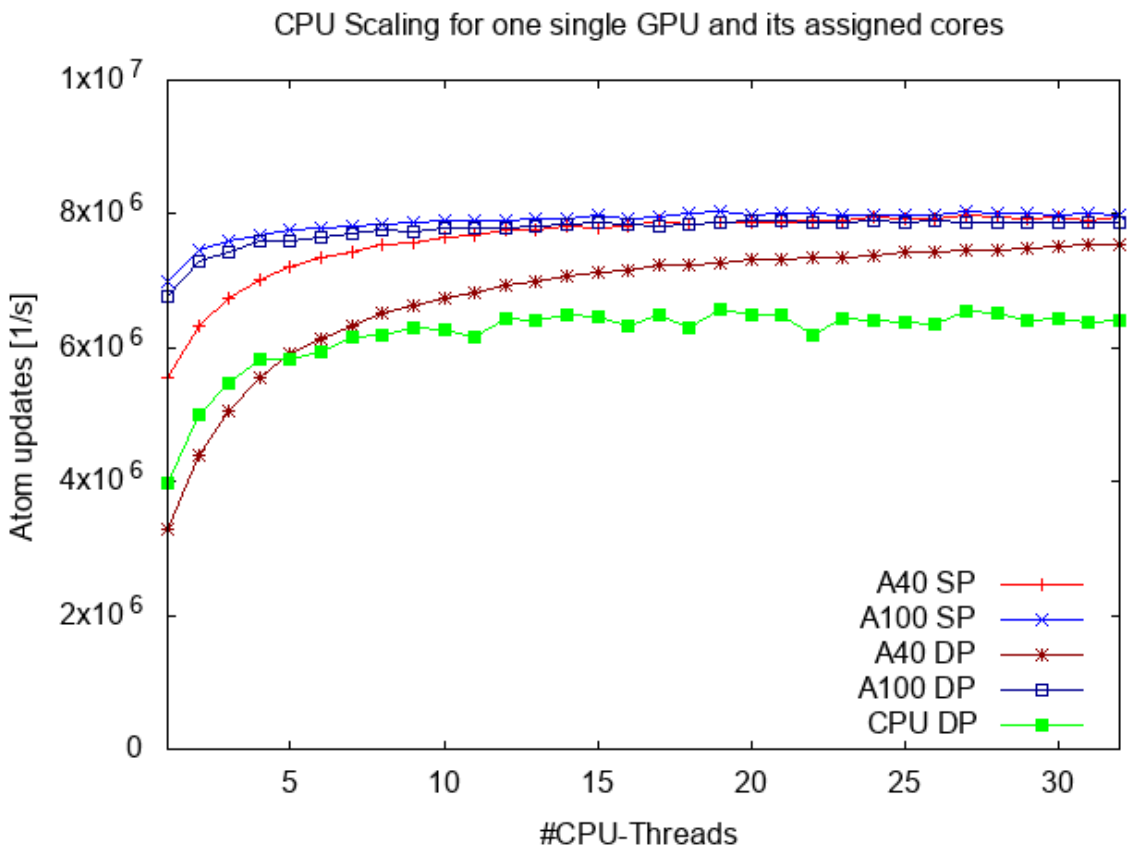
- fixed: one gpu with its associated 16 cores
- scale the number of CPU-Threads
- take output of our programm contains line displaying throughput in number of atom updates per second:

```
Performance: 7.18 million atom updates per second
```

Profiling

Initial: CPU Scaling runs - continued

- plot data for single- and double precision performance



Similar convergence points in performance between the A40 and A100 hints at bottlenecks independent of the gpu.

Profiling the application

Profiling the application

Using nsys

```
srun nsys profile -o ./a40_profiling_run
```

- to get this runtime timeline:

- activity is cyclic
- most of the time is spent with only one CPU active
- short blue parts are where the GPU is active

Profiling the application

Finding where the CPU time is spent

- using gprof
  - GPU time is not counted as such here

```
Flat profile:
Each sample counts as 0.01 seconds.
  %      self              total
 time   seconds    calls  ms/call  name
 97.70     3.35       11   306.47  buildNeighbor
  0.87     0.03      201     0.15  updatePbc
  0.58     0.02  2359296     0.00  myrandom
  0.58     0.02       11     1.82  binatoms
  0.29     0.01       11     0.91  setupPbc
  0.00     0.00     1668     0.00  checkCUDAError
  0.00     0.00      426     0.00  getTimeStamp
  0.00     0.00      201     0.00  computeForce
...
```

Profiling the application

using runtime readings inside the program

- in the standard program output
  - tracks total time
  - takes both CPU and GPU into account
  - total runtime spent on the different phases

```
TOTAL 3.47s FORCE 0.21s NEIGH 3.15s REST 0.12s
```

Even when considering CPU and GPU time the neighbor calculation needs most of the time.

Profiling the application

Explanation of runtime profile with program structure

Recall code from earlier:

```
for t in timesteps:
    GPU-parallel-for atom in atoms:
        velocity[atom] += force[atom]
        position[atom] += velocity[atom]
    if t % 20 == 0:
        neighbors = calculateNeighbors()
    GPU-parallel-for atom in atoms:
        neighbors = neighbors[atom]
        force = 0.0
        for neighbor in neighbors:
            radius = calc_radius(...)
            if radius < close_enough:
                force += calc_force(...)
                forces[atom] += force
```

Parallelizing neighborhood calculation

neighborhood calculation process

- real application simulates a 3D space
- examples written in pseudo-python assume a 2D space for simplicity and brevity

```
def calculate_neighbors():
  update_according_to(atoms, periodic_boundary_condition)
  setup_periodic_boundary_condition(atoms, periodic_boundary_condition)
  update_periodic_boundary_condition(atoms, periodic_boundary_condition)
  neighbor_list = build_neighbor(atoms)
return neighbor_list
```

Parallelizing neighborhood calculation

Atom update

Atoms leaving the simulated area will enter on the mirrored side. In code it looks like this:

```
def update_according_to(atoms, periodic_boundary_condition):
  sim_width, sim_height = periodic_boundary_condition.simulated_area.dims
  x_max, x_min, y_max, y_min = periodic_boundary_condition.boundaries
  for atom in atoms:
    if position[atom].x > x_max:
      position[atom].x -= sim_width
    if position[atom].x < x_min:
      position[atom].x += sim_width
    if position[atom].y > y_max:
      position[atom].y -= sim_height
    if position[atom].y < y_min:
      position[atom].y += sim_height
```

- all iterations independent -> easy to parallelize
- one thread per atom for simplicity (GPU threads are quite lightweight)

Parallelizing neighborhood calculation

Ghost atoms for atoms near periodic border

Create ghost atoms on other side to emulate force interactions through the periodic border

```
def setup_periodic_boundary_condition(atoms, periodic_boundary_condition):
  ...
  n_ghosts = 0,   n_ghost_allocations = some_value
  ghosts = malloc(n_ghost_allocations * sizeof(ghost))
  for atom in atoms:
    if n_ghost_allocations - n_ghosts < max_ghosts_created_per_iteration:
      ghosts = reallocMore(ghosts, some_more)
      n_ghost_allocations += some_more
    if position[atom].x > x_max - threshold:
      ghosts[n_ghosts++] = createGhost(atom, -sim_width, 0)
    ...
    if position[atom].x < x_min + threshold AND position[atom] < y_min + thresh
      ghosts[n_ghosts++] = createGhost(atom, +sim_width, +sim_height)

  if n_atoms + n_ghosts > len(atoms):
    atoms = realloc(n_atoms + n_ghosts)
```

Parallelizing neighborhood calculation

Updating the ghost atom positions

```
def update_bondary_condition(atoms, periodic_boundary_condition):
  for ghost in ghosts:
    position[ghost].x = ghost.offset_x + position[ghost.original].x
    position[ghost].y = ghost.offset_y + position[ghost.original].y
```

- iterations independent
  - easy to parallelize -> one thread per atom

Parallelizing neighborhood calculation

Building neighbor lists

```
def buildNeighbor(atoms):
  ...
  grid_cells = sort_into(atoms),    neighbor_list = malloc(...)
  resize_needed = true
  do:
    new_max_neighbors_per_atom = max_neighbors_per_atom
    resize_needed = false
    for atom in atoms:
      neighbor_count = 0,    grid_cell = grid_cell_of(position[atom])
      for other_atom in (neighboring_cells(grid_cell) + grid_cell).atoms:
        if other_atom == atom: continue
        if distance(atom, other_atom) < threshold:
          neighbor_list[atom, neighbor_count++] = other_atom
      if neighbor_count > max_neighbors_per_atom:
        resize_needed = true
        if neighbor_count > new_max_neighbor_count_per_atom:
          new_max_neighbor_count_per_atom = neighbor_count
    if resize_needed:
      max_neighbors_per_atom = new_max_neighbors_per_atom * 1.2
      free(neighbor_list),      neighbor_list = malloc(...)
  while(resize_needed)
```

Next goals

- port the buildNeighbor-function to Cuda
  - reduces Host-To-Device traffic
  - reduces computation time
- port function updating the ghost atoms to Cuda
- port creating the ghost atoms to Cuda

- evaluate performance along the way

Any questions?

Any other ideas for parallelization?